Breadth-First Search (BFS) - Complete Course

> **Level:** Intermediate (comfortable with data structures) **Language:** Python

---

Table of Contents

1. What is BFS?
2. Core Intuition - The Queue
3. BFS on a Graph
4. BFS on a Tree
5. BFS on a 2D Grid
6. Finding Shortest Paths
7. Multi-Source BFS
8. Bidirectional BFS
9. Common Patterns & Problem Templates
10. Time & Space Complexity
11. BFS vs DFS - When to Use Which
12. Practice Problems

---

1. What is BFS?

Breadth-First Search (BFS) is a graph/tree traversal algorithm that explores nodes **level by level** - visiting all neighbors of a node before moving deeper.

Think of it like ripples expanding outward from a stone dropped in water. Every node at distance 1 is visited before any node at distance 2, and so on.

Key Properties

- Explores nodes in order of their **distance from the source**
- Guaranteed to find the **shortest path** in an unweighted graph
- Uses a **queue** (FIFO) as its core data structure

Visual Example

```
Graph:
    A
   / \
  B   C
 / \   \
D   E   F

BFS order starting from A:
```

```
Level 0: A
Level 1: B, C
Level 2: D, E, F
```

---

2. Core Intuition - The Queue

The secret to BFS is the **queue**. A queue processes nodes in the order they were added (First In, First Out).

The algorithm: 1. Add the starting node to the queue and mark it visited 2. While the queue is not empty: - Dequeue the front node - Process it - Enqueue all unvisited neighbors and mark them visited

Why mark visited BEFORE enqueuing?

This is a common mistake. You must mark a node visited **when you enqueue it**, not when you dequeue it. Otherwise the same node gets added to the queue multiple times, causing redundant work or infinite loops.

```python
#  Wrong - marks visited when dequeuing
queue.append(start)
while queue:
    node = queue.popleft()
    visited.add(node)           # Too late! Node may be enqueued multiple times
    for neighbor in graph[node]:
        if neighbor not in visited:
            queue.append(neighbor)

#  Correct - marks visited when enqueuing
visited.add(start)
queue.append(start)
while queue:
    node = queue.popleft()
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)   # Mark immediately
            queue.append(neighbor)
```

---

3. BFS on a Graph

Setup

Graphs are typically represented as an **adjacency list** - a dictionary mapping each node to its list of neighbors.

```python
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
}
```

Basic BFS Implementation

```python
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)

    result = []

    while queue:
        node = queue.popleft()       # Dequeue from the front
        result.append(node)          # Process the node

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result

# Usage
print(bfs(graph, 'A'))
# Output: ['A', 'B', 'C', 'D', 'E', 'F']
```

Handling Disconnected Graphs

If the graph is not fully connected, a single BFS from one node won't reach all nodes. Iterate over all nodes to cover every component:

```python
def bfs_full(graph):
    visited = set()
    all_components = []

    for node in graph:
```

```
        if node not in visited:
            component = []
            queue = deque([node])
            visited.add(node)

            while queue:
                curr = queue.popleft()
                component.append(curr)
                for neighbor in graph[curr]:
                    if neighbor not in visited:
                        visited.add(neighbor)
                        queue.append(neighbor)

            all_components.append(component)

    return all_components
```

---

4. BFS on a Tree

Trees are a special case of graphs - they have no cycles, so you often don't need a `visited` set (though it doesn't hurt to have one).

Level-Order Traversal

The most common tree BFS pattern processes nodes **level by level**, which is useful when you need to know what level/depth a node is at.

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def level_order(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)   # Number of nodes at this level
        level = []
```

```
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level)

    return result

# Example tree:     1
#                  / \
#                 2   3
#                / \
#               4   5
#
# Output: [[1], [2, 3], [4, 5]]
```

Key Pattern: `level_size = len(queue)`

Capturing `len(queue)` at the start of each outer loop iteration is the standard trick to process nodes level by level. It tells you exactly how many nodes belong to the current level before you start adding the next level's nodes.

---

5. BFS on a 2D Grid

Grids are one of the most frequent BFS problem types. Each cell `(r, c)` is a node, and its neighbors are the adjacent cells (up, down, left, right - sometimes diagonals too).

Template

```
from collections import deque

def bfs_grid(grid, start_r, start_c):
    rows, cols = len(grid), len(grid[0])
    visited = set()
    queue = deque()

    visited.add((start_r, start_c))
    queue.append((start_r, start_c))
```

```
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]  # right, left, down, up

    while queue:
        r, c = queue.popleft()
        # Process cell (r, c) here

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and              # Within bounds
                0 <= nc < cols and              # Within bounds
                (nr, nc) not in visited and     # Not visited
                grid[nr][nc] != '#'):           # Not a wall (adjust condition)
                visited.add((nr, nc))
                queue.append((nr, nc))
```

Example: Number of Islands

Count the number of islands in a binary grid (`'1'` = land, `'0'` = water).

```
def num_islands(grid):
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    visited = set()
    islands = 0

    def bfs(r, c):
        queue = deque([(r, c)])
        visited.add((r, c))
        while queue:
            row, col = queue.popleft()
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = row + dr, col + dc
                if (0 <= nr < rows and 0 <= nc < cols
                        and grid[nr][nc] == '1'
                        and (nr, nc) not in visited):
                    visited.add((nr, nc))
                    queue.append((nr, nc))

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                bfs(r, c)
                islands += 1
```

```
    return islands
```

---

6. Finding Shortest Paths

BFS naturally finds the **shortest path** in an unweighted graph because it explores nodes in order of distance.

Shortest Path - Distance Only

```
from collections import deque

def shortest_distance(graph, start, end):
    if start == end:
        return 0

    visited = {start}
    queue = deque([(start, 0)])   # (node, distance)

    while queue:
        node, dist = queue.popleft()

        for neighbor in graph[node]:
            if neighbor == end:
                return dist + 1
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))

    return -1   # No path found
```

Shortest Path - Reconstruct the Actual Path

To reconstruct the path, store a `parent` map that records how you reached each node.

```
from collections import deque

def shortest_path(graph, start, end):
    visited = {start}
    queue = deque([start])
    parent = {start: None}   # Maps each node to the node it came from

    while queue:
        node = queue.popleft()
```

```
        if node == end:
            break
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = node
                queue.append(neighbor)

    # Reconstruct path by walking back through parent map
    if end not in parent:
        return []   # No path

    path = []
    curr = end
    while curr is not None:
        path.append(curr)
        curr = parent[curr]

    return path[::-1]   # Reverse to get start -> end

# Example
graph = {'A': ['B','C'], 'B': ['A','D'], 'C': ['A','D'], 'D': ['B','C']}
print(shortest_path(graph, 'A', 'D'))
# Output: ['A', 'B', 'D'] or ['A', 'C', 'D']
```

---

7. Multi-Source BFS

Sometimes you need to find the shortest distance from **multiple starting points** simultaneously - for example, "distance to the nearest exit" or "distance to the nearest 0 in a grid."

The trick: **add all sources to the queue at the start**, all with distance 0. BFS then naturally expands from all of them simultaneously.

Example: 01 Matrix (distance to nearest 0)

```
from collections import deque

def update_matrix(mat):
    rows, cols = len(mat), len(mat[0])
    dist = [[float('inf')] * cols for _ in range(rows)]
    queue = deque()

    # Seed queue with all 0-cells (the sources)
    for r in range(rows):
```

```
        for c in range(cols):
            if mat[r][c] == 0:
                dist[r][c] = 0
                queue.append((r, c))

    # BFS expands outward from all 0s simultaneously
    while queue:
        r, c = queue.popleft()
        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and dist[nr][nc] > dist[r][c] + 1):
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))

    return dist
```

---

8. Bidirectional BFS

For finding the shortest path between two specific nodes, **bidirectional BFS** can be dramatically faster. Instead of searching from just the start, you search from **both** the start and the end simultaneously, meeting in the middle.

- Regular BFS explores O(b^d) nodes (b = branching factor, d = depth)
- Bidirectional BFS explores O(b^(d/2)) nodes from each side - exponentially faster

```
from collections import deque

def bidirectional_bfs(graph, start, end):
    if start == end:
        return 0

    # Two frontiers
    front = {start}
    back = {end}
    visited_front = {start}
    visited_back = {end}
    dist = 0

    while front and back:
        dist += 1

        # Always expand the smaller frontier
        if len(front) > len(back):
```

```
            front, back = back, front
            visited_front, visited_back = visited_back, visited_front

        next_front = set()
        for node in front:
            for neighbor in graph[node]:
                if neighbor in visited_back:
                    return dist         # The two frontiers met!
                if neighbor not in visited_front:
                    visited_front.add(neighbor)
                    next_front.add(neighbor)
        front = next_front

    return -1   # No path
```

---

9. Common Patterns & Problem Templates

Pattern 1: BFS with State

When the "node" in your BFS isn't just a location but a combination of location + state (e.g., position + number of keys collected), encode the full state as the queue element.

```
# Example: shortest path where you can break at most k walls
from collections import deque

def shortest_path_with_k_breaks(grid, k):
    rows, cols = len(grid), len(grid[0])
    # State: (row, col, remaining_breaks)
    start = (0, 0, k)
    queue = deque([(start, 0)])     # (state, distance)
    visited = {start}

    while queue:
        (r, c, breaks), dist = queue.popleft()

        if r == rows - 1 and c == cols - 1:
            return dist

        for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                new_breaks = breaks - (1 if grid[nr][nc] == 1 else 0)
                state = (nr, nc, new_breaks)
                if new_breaks >= 0 and state not in visited:
```

```
                    visited.add(state)
                    queue.append((state, dist + 1))

    return -1
```

Pattern 2: BFS on Implicit Graphs

Some problems don't give you an explicit graph - the graph is defined by rules (e.g., word ladder: each word connects to words that differ by one letter).

```
# Word Ladder: minimum steps to transform begin_word -> end_word
from collections import deque

def word_ladder(begin_word, end_word, word_list):
    word_set = set(word_list)
    if end_word not in word_set:
        return 0

    queue = deque([(begin_word, 1)])
    visited = {begin_word}

    while queue:
        word, steps = queue.popleft()

        for i in range(len(word)):
            for ch in 'abcdefghijklmnopqrstuvwxyz':
                new_word = word[:i] + ch + word[i+1:]
                if new_word == end_word:
                    return steps + 1
                if new_word in word_set and new_word not in visited:
                    visited.add(new_word)
                    queue.append((new_word, steps + 1))

    return 0
```

---

10. Time & Space Complexity

Scenario	Time	Space
BFS on graph (V vertices, E edges)	O(V + E)	O(V)
BFS on grid (R rows, C cols)	O(R x C)	O(R x C)
BFS on tree (N nodes)	O(N)	O(N) - worst case, wide tree
Bidirectional BFS	O(b^(d/2))	O(b^(d/2))

Space breakdown: - The `queue` holds at most O(V) nodes - The `visited` set

holds at most O(V) nodes - The `parent` map (for path reconstruction) holds at most O(V) entries

---

11. BFS vs DFS - When to Use Which

Use BFS when...	Use DFS when...
You need the **shortest path**	You need **any** path
The graph is **wide but shallow**	The graph is **narrow but deep**
You need **level-by-level** processing	You need **topological sort**
Finding **nearest** neighbor/exit	Detecting **cycles**
Multi-source spreading problems	**Backtracking** problems

Rule of thumb: If the problem involves "minimum steps", "fewest moves", or "nearest X" in an unweighted graph - reach for BFS first.

---

12. Practice Problems

Work through these in order. Each one reinforces a specific BFS pattern.

Beginner

#	Problem	Pattern
1	Binary Tree Level Order Traversal	Basic tree BFS
2	Flood Fill	Basic grid BFS
3	Number of Islands	Grid BFS + connected components

Intermediate

#	Problem	Pattern
4	Rotting Oranges	Multi-source BFS
5	01 Matrix	Multi-source BFS
6	Walls and Gates	Multi-source BFS
7	Word Ladder	Implicit graph BFS
8	Shortest Path in Binary Matrix	Grid BFS + shortest path

Advanced

#	Problem	Pattern
9	Shortest Path with K Obstacles	BFS with state
10	Open the Lock	BFS with state
11	Word Ladder II (all shortest paths)	BFS + backtracking
12	Cut Off Trees for Golf Event	Multi-BFS

Quick Reference Cheatsheet

```python
from collections import deque

# --- Graph BFS ---
def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# --- Tree Level Order ---
def level_order(root):
    queue = deque([root])
    while queue:
        for _ in range(len(queue)):   # Process one level
            node = queue.popleft()
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)

# --- Grid BFS ---
DIRS = [(0,1),(0,-1),(1,0),(-1,0)]
def bfs_grid(grid, sr, sc):
    rows, cols = len(grid), len(grid[0])
    visited = {(sr, sc)}
    queue = deque([(sr, sc)])
    while queue:
        r, c = queue.popleft()
        for dr, dc in DIRS:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited:
                visited.add((nr, nc))
                queue.append((nr, nc))
```

```python
# --- Shortest Path with Distance ---
def bfs_dist(graph, start, end):
    visited = {start}
    queue = deque([(start, 0)])
    while queue:
        node, dist = queue.popleft()
        if node == end: return dist
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist+1))
    return -1
```

Happy coding! BFS is one of those algorithms that, once it clicks, you start seeing it everywhere.